

Lab #1 Pthreads and OpenMP

22.04.2026

This laboratory exercise will introduce the programming of shared memory systems using pthreads and OpenMP.

Pthreads and OpenMP both enable parallelism on shared memory architectures but differ in their level of abstraction. Pthreads provide fine-grained control over thread management and synchronization, often requiring substantial modifications to the source code. In contrast, OpenMP offers a higher-level, directive-based approach that simplifies parallelization while achieving similar performance goals.

Pthreads

Parallelizing a multiply accumulate with runtime constraint

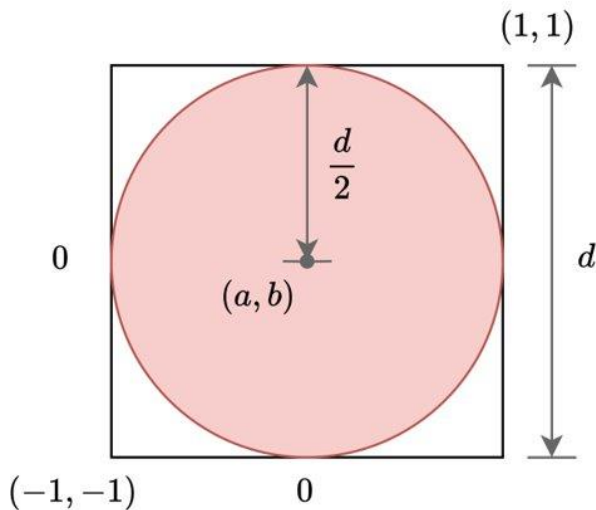
Navigate to pthreads/00_multiply_accumulate, and open the main.c. The program takes two arrays of floating point numbers and computes the running sum based on a runtime condition. Since there are no dependencies between the array elements, the task can be split across multiple threads. The code provided is missing thread initialization and synchronization for access global variables.

1. Add code for creating NUM_THREADS, and ensure that the main thread waits for all worker threads to complete execution.
2. Introduce synchronization mechanisms to prevent concurrent access to the global sum variable.

Taking advantage of the pre-processor directives, compile the program using different number of threads (e.g. 1, 2, 4 ... 16), noting the sum and wall clock time. What can you conclude from these observations?

Estimating π with a Montecarlo simulation

Monte Carlo simulations estimate outcomes by performing repeated random experiments. In this exercise, we will use the Monte Carlo method to estimate the value of π . The estimation is based on comparing the area of a circle with radius $d/2$ to the area of a square with side length d . With reference to Figure 1, if the square fully encloses the circle and random points are uniformly sampled within the square, the ratio of points that fall inside the circle to the total number of samples approximates $\pi/4$.



$$A_c = \pi \left(\frac{d}{2} \right)^2 \quad (1)$$

$$A_s = d^2 \quad (2)$$

$$\frac{A_c}{A_s} = \frac{\pi}{4} \quad (3)$$

$$(x - a)^2 + (y - b)^2 = \left(\frac{d}{2} \right)^2 \quad (4)$$

Figure 1. Monte Carlo simulation for estimating π , showing the geometric setup on the left and key equations on the right.

The directory `pthread/01_monte_carlo_single_threaded` contains a single-threaded reference implementation of this simulation. Parallelize the core computation with Pthreads ensuring proper synchronization where appropriate. What's the estimate of π after running the simulation for at least $1e10$ cycles?

OpenMP – Matrix Multiplication

In this section we will use OpenMP to easily parallelize a matrix-matrix multiplication and analyze its characteristics. Navigate to `openmp/01_matrix_multiply`, and open the `main.c`. Skim through the source code and understand how the multiplication of two arbitrary matrices is set up and performed.

Compile the code and run it to establish a baseline. Afterwards, add the necessary OpenMP compiler directives to parallelize the multiplication loop. Using the provided script, `exp.sh` or otherwise sweep the number of threads and size of matrices used over a range that is manageable on your machine. At what point does the problem shift from being compute-bound to memory-bound?